We have updated the project's target platform from iOS and Apple Watch to Android and a compatible smartwatch to better align with the devices available to our team and support prototype development and testing within the project timeline.

Functional Requirements

1- User Registration

The system shall allow users to create an account using an email address and password.

2- User Login and Logout

The system shall allow registered users to log in using their email address and password and securely log out.

3- Password Recovery

The system shall allow users to reset their password through their registered email address.

4- Personal and Health Profile

The system shall allow users to enter and update their name, date of birth, supported medical conditions, and relevant medications. The system shall calculate the user's age from their date of birth.

5- User Consent and Permissions

The system shall obtain the user's consent before collecting health data and request the permissions required for heart rate monitoring and notifications.

6- Smartwatch Connection

The system shall communicate with a supported smartwatch paired with the Android phone and display its connection status.

7- Automatic Heart Rate Collection

The system shall automatically receive new heart rate readings and their measurement timestamps from the connected smartwatch, without requiring users to start a monitoring session or manually upload files.

8- AI-Based Heart Rate Classification

The system shall use a trained and evaluated machine learning model to classify valid heart rate data as High, Low, or Normal within the project's defined scope.

9- Alert Generation and Notifications

The system shall generate an alert when valid heart rate readings meet predefined alert conditions, considering their persistence and available contextual information. It shall display a phone notification that opens the alert details and related instructions when selected.

10- Symptom Reporting

The system shall allow users to report symptoms associated with an alert and use this information when selecting guidance according to predefined rules.

11- Personalized Health Instructions

The system shall display general health instructions selected according to the alert type, reported symptoms, and relevant profile information, based on documented health sources.

Restricted - مقيد

12- Personal Dashboard

The system shall provide a dashboard displaying the latest valid heart rate reading, its measurement time, its classification, the smartwatch connection status, and a heart rate trend chart.

13- Heart Rate and Alert History

The system shall allow users to view their previous heart rate readings and alerts, including timestamps, classifications, and associated instructions.

14- Data Storage

The system shall store each user’s health profile, heart rate readings, classification results, and alert history under that user’s account.

15- Connection Loss and Recovery

The system shall indicate when the smartwatch connection is lost or readings become outdated, attempt to reconnect, and resume receiving readings when the connection is restored.

16- Account and Data Deletion

The system shall allow users to request deletion of their account and associated personal and health data, with confirmation before deletion.

Restricted - مقيد

Project Tasks and Team Responsibilities

Task ID	Task Name	Subtasks	Deliverable	Assigned To	Due Date	Status
T1	Requirements Analysis & Platform Update **(Revised)**	1- Review and finalize functional and non-functional requirements. 2- Define target users, use of medical history, application limitations, and monitoring mode. 3- Update the proposal and SP-I report from Apple Watch to Android / Wear OS (Galaxy Watch) and state the reason for the change.	Finalized requirements, project scope, and updated proposal/report.	Nofe		Done
T2	Smartwatch Connectivity & Background Monitoring **(Revised)**	1- Implement a watch app to request permissions and read heart rate. 2- Transfer heart rate readings and timestamps to the phone. 3- Implement continuous background monitoring when the app is closed or the watch screen is off. 4- Test connection recovery, battery use, and data transfer delay.	Working prototype that transfers real heart rate readings continuously in the background.	Nofe		Done
T3	UI/UX Design	1- Define user flows and screen navigation. 2- Design phone screens and the basic watch display. 3- Design loading, missing-data, connection, and error states. 4- Build and review an interactive prototype.	Complete interface designs and an interactive navigation prototype.	Nofe		Done
T4	User Accounts and Health Profile	1- Implement registration, login, logout, and password recovery. 2- Implement personal and health profile creation and editing. 3- Implement consent recording and permission requests. 4- Implement account and associated data deletion with confirmation.	Working user accounts and editable health profiles, including consent and account deletion features.	Rawan		Done
T5	Database Development	1- Design the database schema and relationships. 2- Create storage for profiles, readings, classifications, alerts, and instructions. 3- Implement user-specific access controls and secure data communication. 4- Test data saving, retrieval, updating, and deletion.	Implemented database with a schema diagram and tested access controls.	Rawan		Done

Restricted - مقيد

Task ID	Task Name	Subtasks	Deliverable	Assigned To	Due Date	Status
T6	Public Dataset Acquisition & Exploratory Analysis **(NEW)**	1- Download the Galaxy Watch Active2 4-week dataset (Figshare, doi:10.6084/m9.figshare.28509740) and GalaxyPPG (Zenodo, record 14635823). 2- Explore each dataset: fields, sampling rate, participants, missing values, and heart rate distributions. 3- Analyze heart rate against activity level, steps, and time of day to confirm the context features. 4- Align both datasets to a common resolution (e.g., 5-minute windows) and document licenses and limitations.	Exploratory data analysis (EDA) report with cleaned, aligned public datasets.	Lyan, Abrar		Not started
T7	Self-Collected Data Collection **(NEW)**	1- Prepare a participant consent form and data collection protocol. 2- Collect heart rate data using the team's Galaxy Watch7 and the T2 app for at least 2 weeks per member. 3- Record context tags (rest, walking, exercise, sleep) with each session. 4- Store, anonymize, and document the collected data.	Anonymized self-collected Galaxy Watch7 dataset with consent forms and a collection report.	Lyan, Nofe		Not started
T8	Data Processing & Feature Extraction **(Revised)**	1- Validate readings and handle missing, invalid, or duplicate data. 2- Extract features per time window: mean HR, SD, deviation from personal baseline, steps/activity level, time of day, and abnormal duration. 3- Apply the same pipeline to public data, self-collected data, and live app readings.	Shared data-processing pipeline producing model-ready features.	Rehab		Not started
T9	Evaluation Set & Labelling **(Revised)**	1- Define abnormal heart rate events using clinical rules plus context (e.g., resting HR > 100 or < 50 bpm for ≥ 10 minutes). 2- Inject synthetic anomalies with known locations into real normal data. 3- Mark normal high-HR activity periods (jogging, running, exercise) from GalaxyPPG and self-collected data to measure false alarms. 4- Split data into training, validation, and testing sets without participant or time-window overlap.	Documented, labelled evaluation dataset with train/validation/test splits.	Abrar		Not started
T10	Baseline & Context-Aware Model (XGBoost / Random Forest) **(NEW)**	1- Implement the fixed-threshold method (> 100 / < 60 bpm) as the baseline. 2- Train XGBoost and Random Forest to predict the expected heart rate from activity, steps, time of day, and personal baseline. 3- Flag anomalies when the actual heart rate deviates significantly from the predicted value; tune the deviation threshold. 4- Explain predictions using feature importance / SHAP.	Trained context-aware model with the baseline implementation and explanation of results.	Rehab		Not started

Restricted - مقيد

Task ID	Task Name	Subtasks	Deliverable	Assigned To	Due Date	Status
T11	Unsupervised Anomaly Detection (Isolation Forest) **(NEW)**	1- Train Isolation Forest on normal data using the extracted features. 2- Build personal models by fine-tuning on the first week of each user's data. 3- Tune contamination and decision thresholds on the validation set.	Trained and tuned Isolation Forest model with personalization results.	Abrar		Not started
T12	Time-Series Model (LSTM Autoencoder) – Advanced **(NEW)**	1- Prepare sequential windows from the Galaxy Watch Active2 data. 2- Train an LSTM Autoencoder on normal sequences. 3- Flag anomalies using reconstruction error and tune the threshold. 4- Implement only after T10 and T11 are completed.	Trained LSTM Autoencoder with results (optional extension).	Rawan, Lyan		Not started
T13	Model Evaluation, Comparison & Integration **(NEW)**	1- Compare all models with the fixed-threshold baseline. 2- Measure Precision, Recall, and F1-score on injected anomalies. 3- Measure the false alarm rate on normal high-HR activity periods. 4- Test on self-collected Galaxy Watch7 data, select the best model, and integrate it into the app. 5- Export the selected model (ONNX / TensorFlow Lite) with scaler parameters (JSON) and verify that Python and Android outputs match on the same samples.	Comparative evaluation report and the selected model integrated into the app.	Rehab, Abrar		Not started
T14	Alerts and Notifications	1- Implement alert conditions based on the model output and persistence rules. 2- Implement symptom reporting and link alerts to relevant instructions.	Tested alert and notification system with symptom reporting and linked instructions.	Rawan		Not started
T15	Personalized Health Instructions	1- Prepare an instruction library using documented health sources (e.g., WHO, AHA, Saudi MOH). 2- Define selection rules using the detected condition, reported symptoms, and relevant profile data.	Reviewed instruction library with tested personalization rules. Rule-based decision table (condition + symptoms + profile → instruction)	Rehab+ Nofe		Not started
T16	Dashboard and History	1- Display the latest valid reading, timestamp, and classification. 2- Implement the heart rate trend chart. 3- Implement reading and alert history with access to event details.	Working dashboard with a heart rate chart and reading and alert history.	Lyan		Not started

Restricted - مقيد

Task ID	Task Name	Subtasks	Deliverable	Assigned To	Due Date	Status
T17	System Integration and Testing	1- Integrate the watch app, phone app, database, model, and alert logic. 2- Test the complete workflow using real readings and clearly labelled simulated scenarios. 3- Test permissions, disconnections, background monitoring, account isolation, and data deletion.	Integrated application with a test report documenting results and remaining issues.	Nofe (lead), Rawan		Not started
T18	Documentation and Final Presentation	1- Update the report to match the implemented system, evaluation results, and limitations. 2- Prepare installation instructions, the user guide, and organized submission files. 3- Prepare and rehearse the presentation, live demonstration, and backup video.	Final report, source code, installation files, user guide, presentation.	All members		Not started

Restricted - مقيد

Task 1: Requirements Analysis and Project Scope

Our project aims to build an app that connects to a smartwatch and automatically receives heart rate readings. The app will classify these readings as High, Low, or Normal and alert users when needed, providing guidance and advice for each situation. Users will also be able to view their readings and history on a personal dashboard.

Target Users

The app is designed for adults aged 18 and above who want to keep track of their heart rate for general health and fitness. Users can enter their medical history to help the app select relevant general guidance.

Supported Health Conditions

The app is designed to support heart rate monitoring, fitness, and general wellness. It is not intended to diagnose medical conditions, provide medical consultations, or recommend treatments for specific diseases. It provides alerts and general guidance on possible next steps based on heart rate readings and relevant information from the user's reported medical history. This guidance does not replace advice or care from a qualified healthcare professional.

Monitoring and Data Collecting

The app will receive heart rate readings automatically from the connected smartwatch, along with the time of each reading. Users will not need to enter the readings themselves or upload any files. If the watch disconnects, the app will let the user know and start receiving new readings again when the connection returns.

Task 2: Smartwatch–Phone Integration Prototype

We built a simple prototype using Android Studio and Kotlin to connect a Galaxy Watch7 to an Android phone. After the user grants permission to read heart rate, the watch app displays the readings and automatically sends the latest reading and its measurement time to the phone through the Wear OS Data Layer API. We tested the prototype with both apps open and confirmed that the readings updated on the phone without any manual input. So far, we have tested this while both apps are open; monitoring when the apps are closed or the watch screen is off is still to be developed.

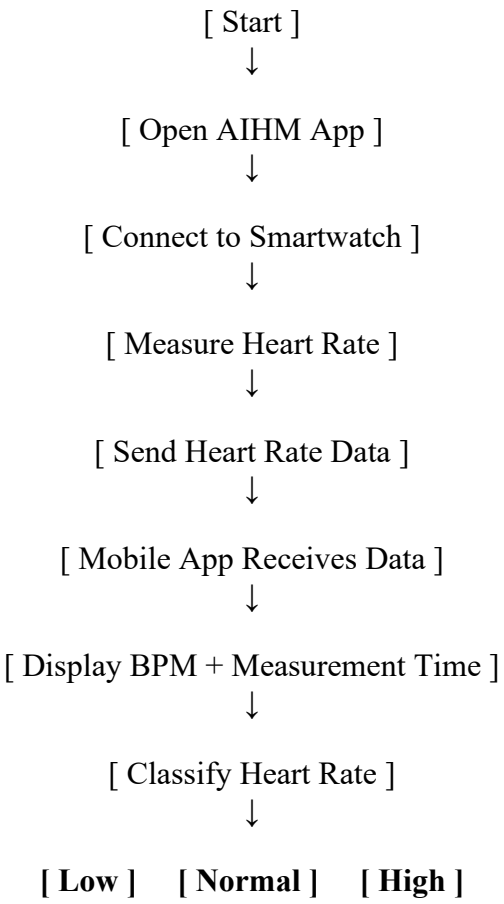
Restricted - مقيد

Figure 1: Heart rate reading on the smartwatch and its synchronized display on the Android phone.



Restricted - مقيد

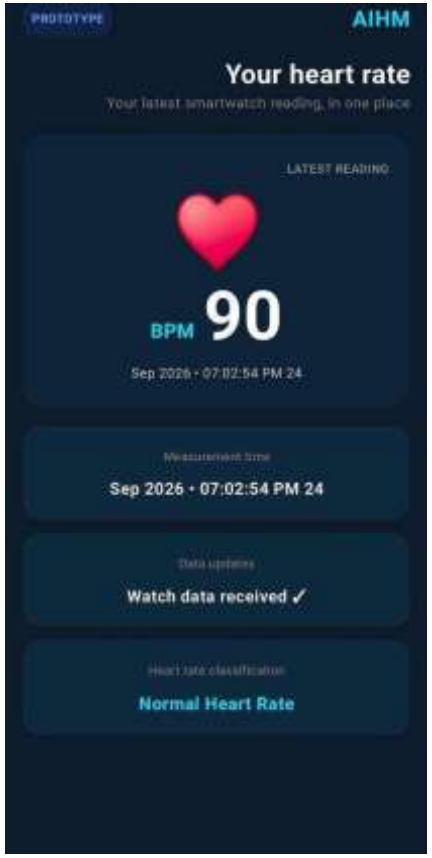
User Flow Diagram



This diagram shows the user flow of the AIHM prototype. The smartwatch measures the user's heart rate and sends the data to the mobile application. The app then displays the heart rate (BPM) and the measurement time, and classifies the reading as Low, Normal, or High.

Restricted - مقيد

Figure 2: AIHM phone app prototype showing the latest heart rate reading and its classification.

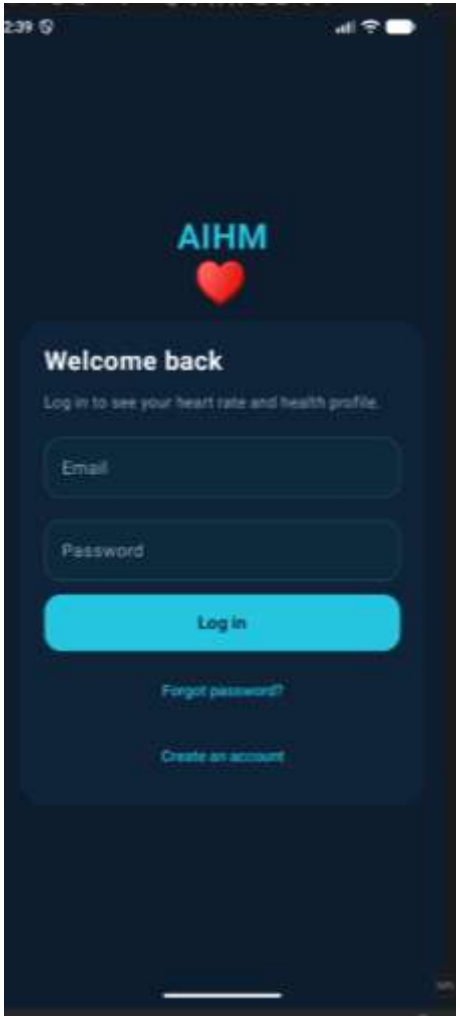


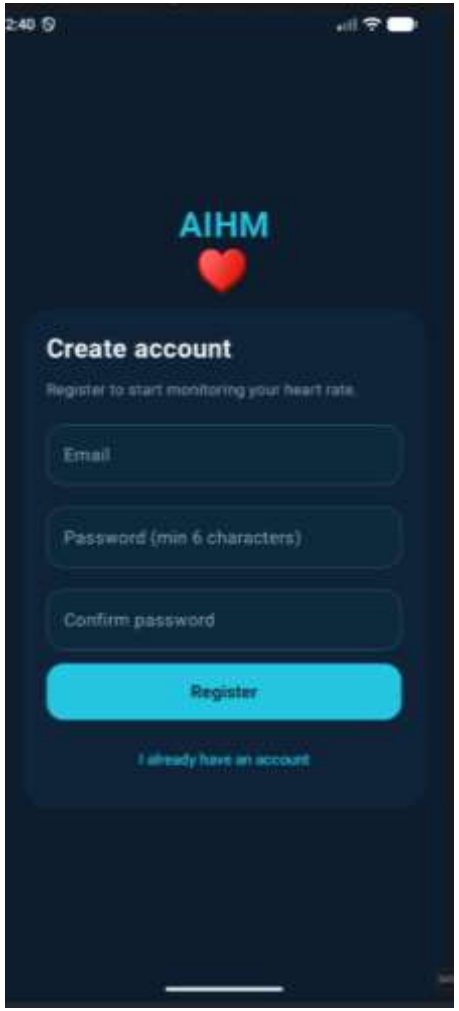
Restricted - مقيد

Task 4: User Accounts and Health Profile

We built the account and profile features of the phone app using Android Studio, Kotlin, and Jetpack Compose. Firebase Authentication handles user accounts, and Cloud Firestore stores each user's data. A new user can register with an email address and password, log in, log out, and reset a forgotten password through their registered email. When the app opens, it checks whether the user is already logged in and shows the login screen only if they are not.

Figure 3: Login and registration screens of the phone app

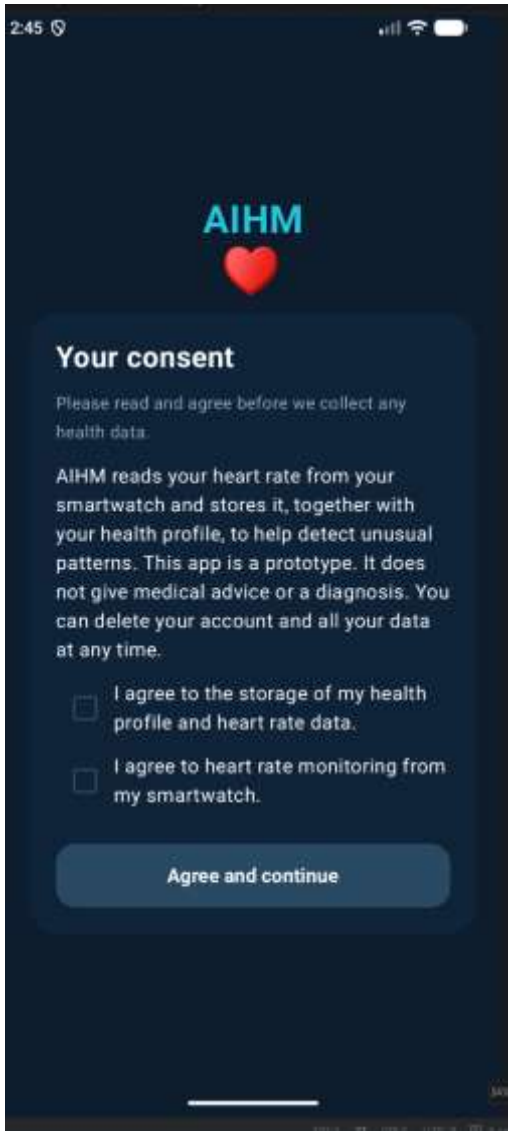


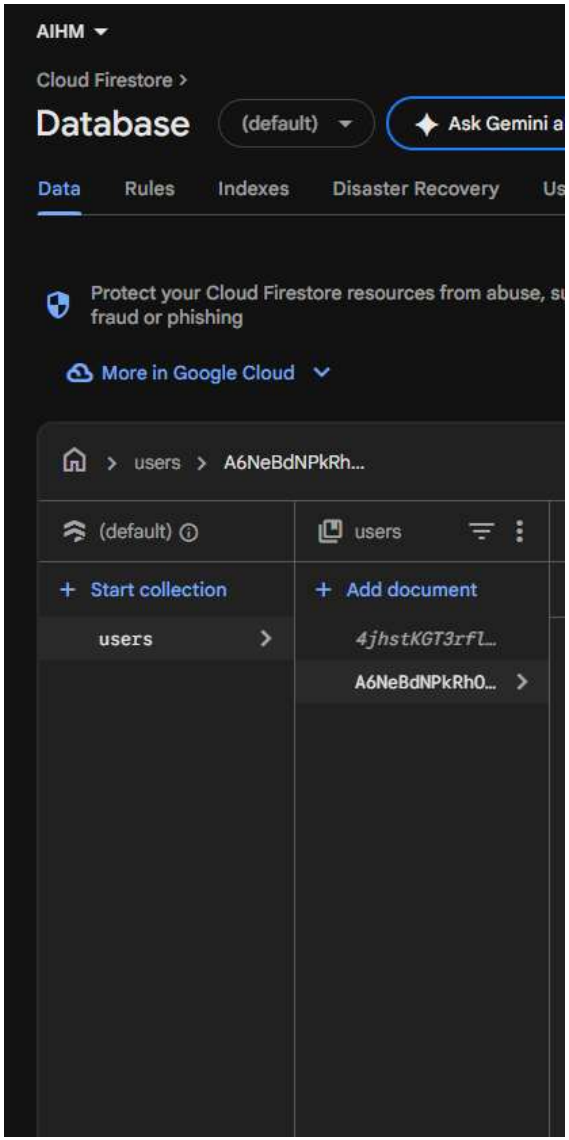


Restricted - مقيد

After registering, the user sees a consent screen that explains what data the app collects and states that the app does not give medical advice or a diagnosis. The user must accept two statements, one for storing health data and one for heart rate monitoring, before continuing. Each agreement is saved with a timestamp under the user's account.

Figure 4: Consent screen and the saved consent records in Cloud Firestore.

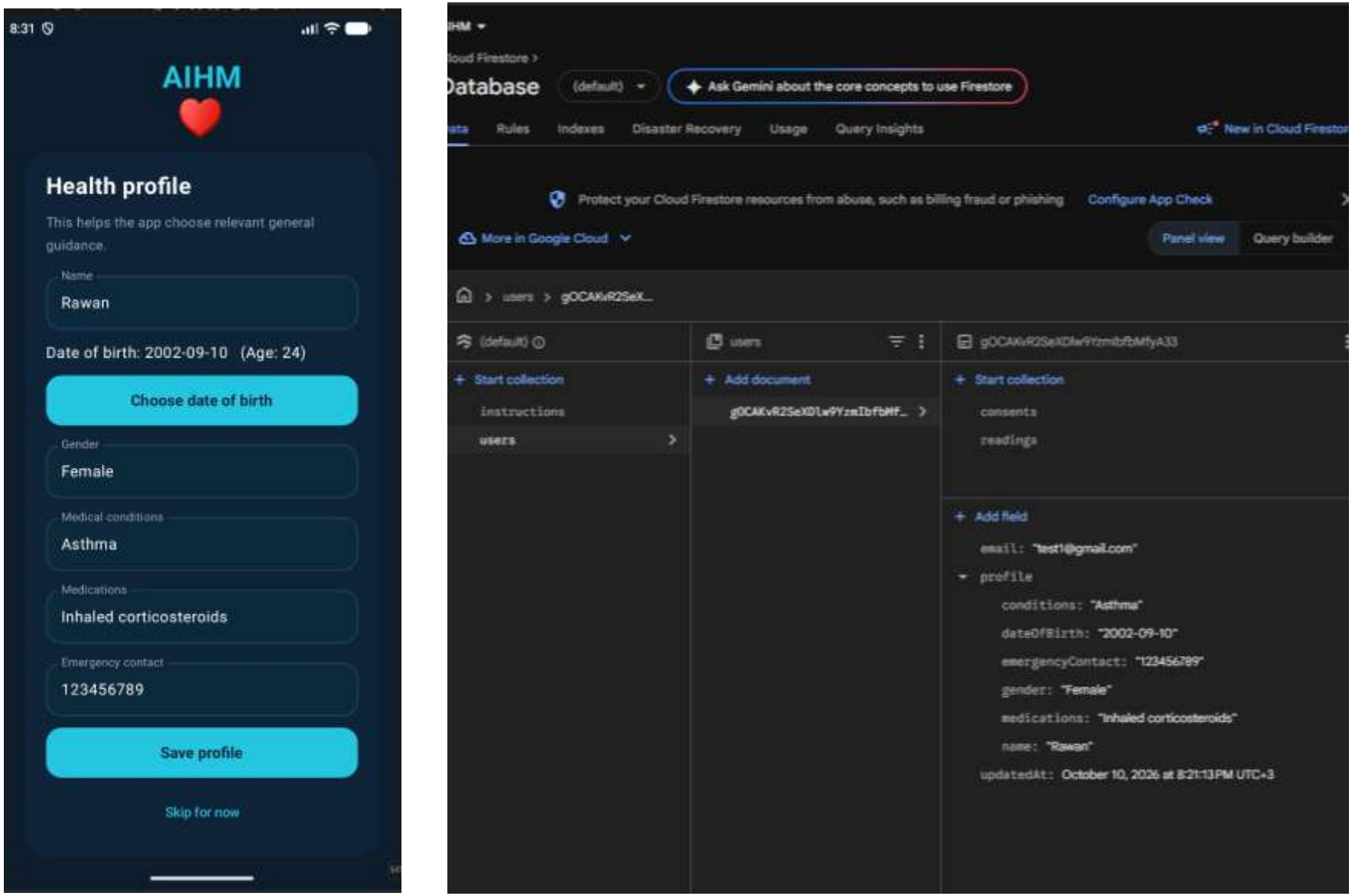




Restricted - مقيد

The user then fills in a health profile with their name, date of birth, gender, medical conditions, medications, and an emergency contact. They can edit the profile later from the dashboard, and the saved values load back automatically.

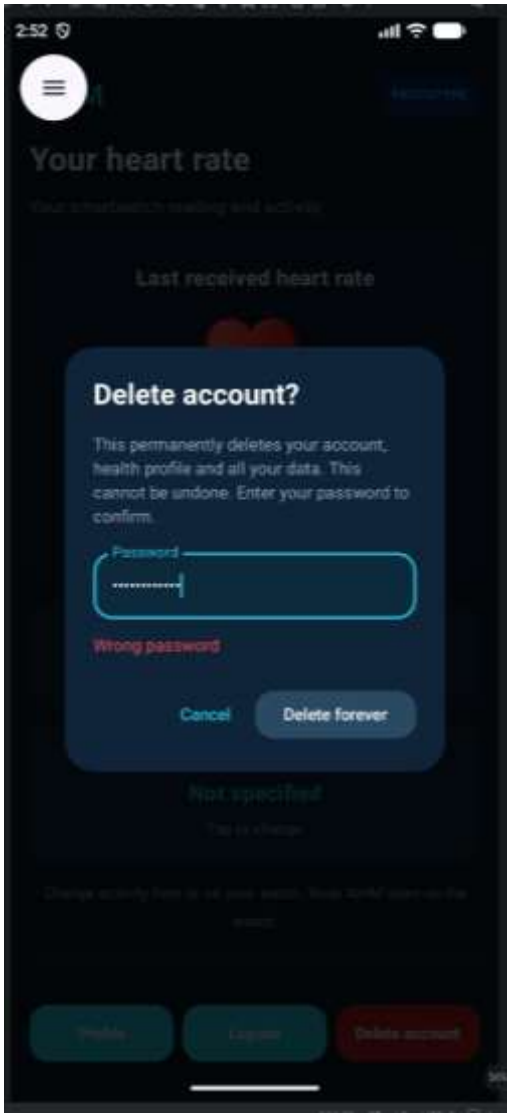
Figure 5: Health profile screen and the saved profile in Cloud Firestore.

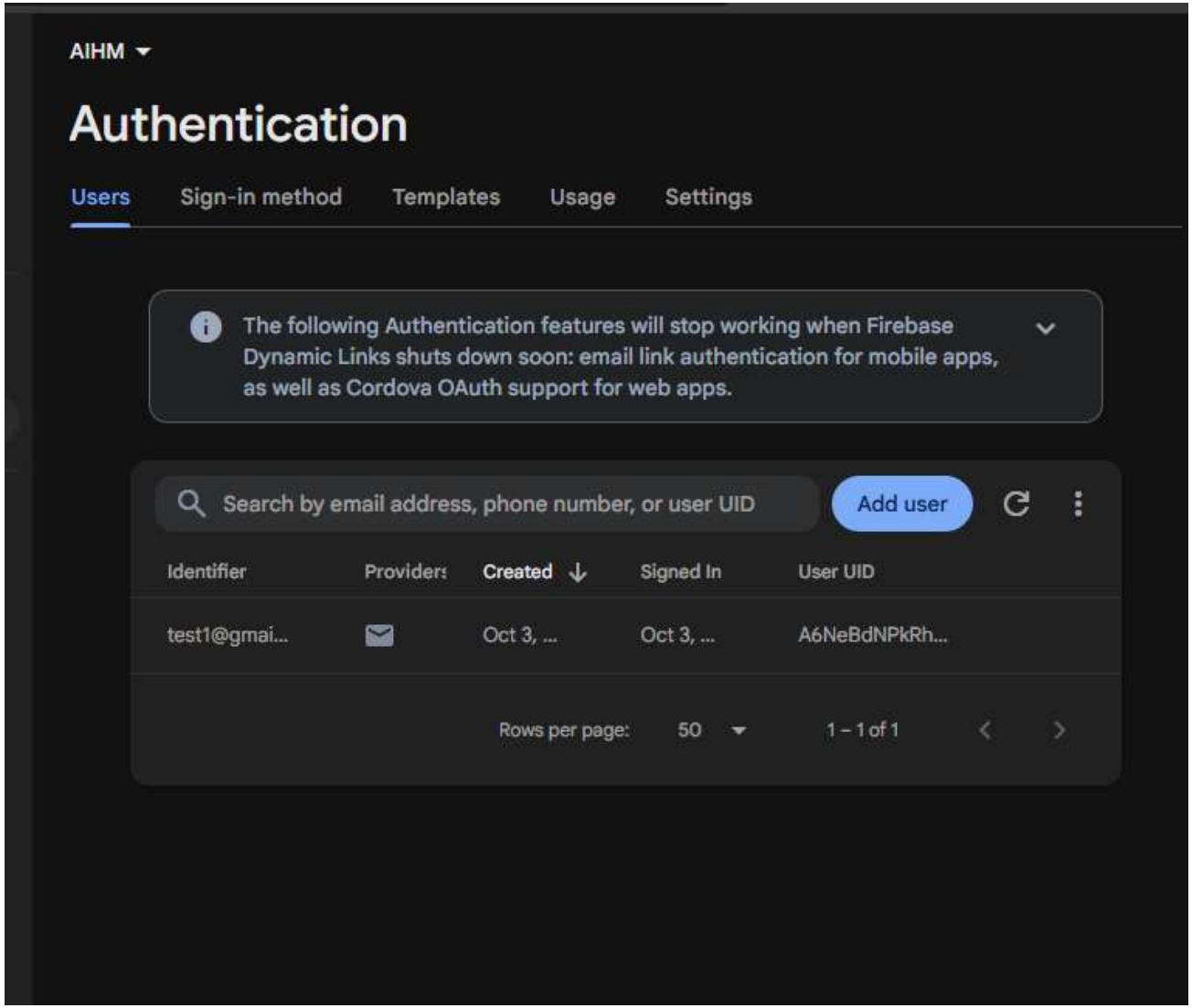


Restricted - مقيد

Users can also delete their account from the dashboard. The app asks for a confirmation and the user's password, then permanently deletes the profile, consent records, heart rate readings, and login account. We tested this with a wrong password, which was rejected, and with the correct password, after which the user disappeared from both Authentication and Firestore.

Figure 6: Account deletion confirmation, the wrong-password check, and the data removed after deletion.

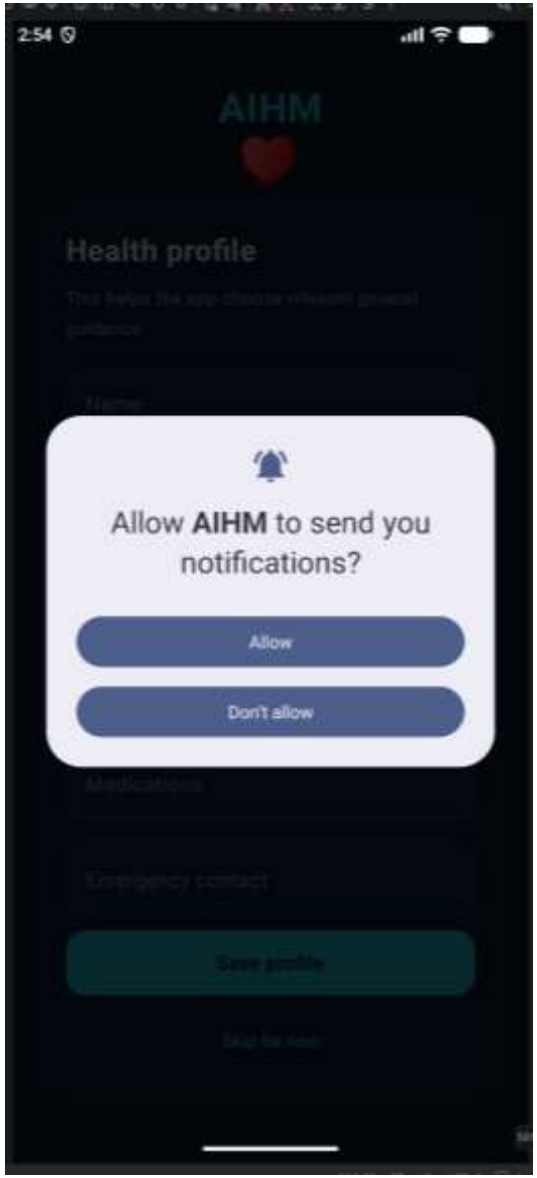




Restricted - مقيد

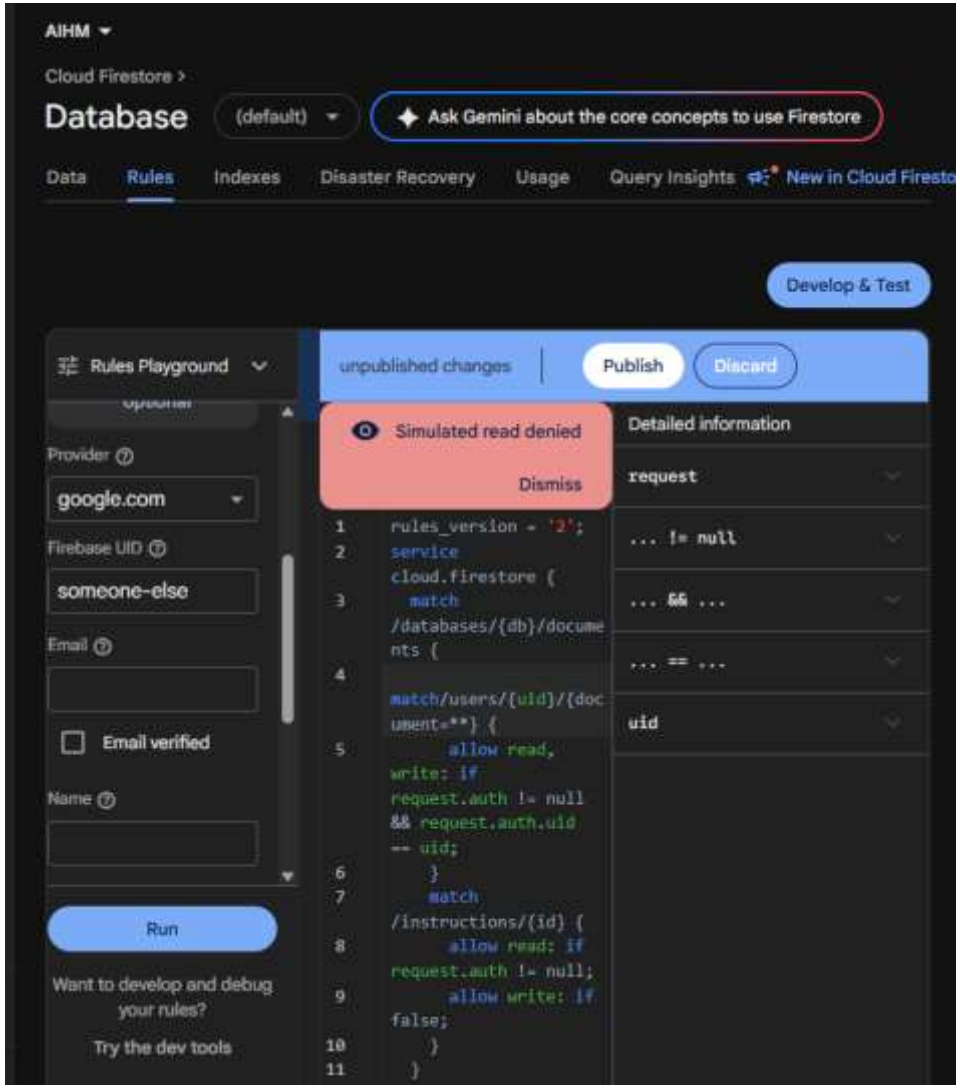
The app requests the notification permission after the user gives consent. The heart rate permission is requested on the smartwatch app. The profile stores the user's date of birth, selected from a calendar, and the app calculates the age from it. Users must be 18 or older.

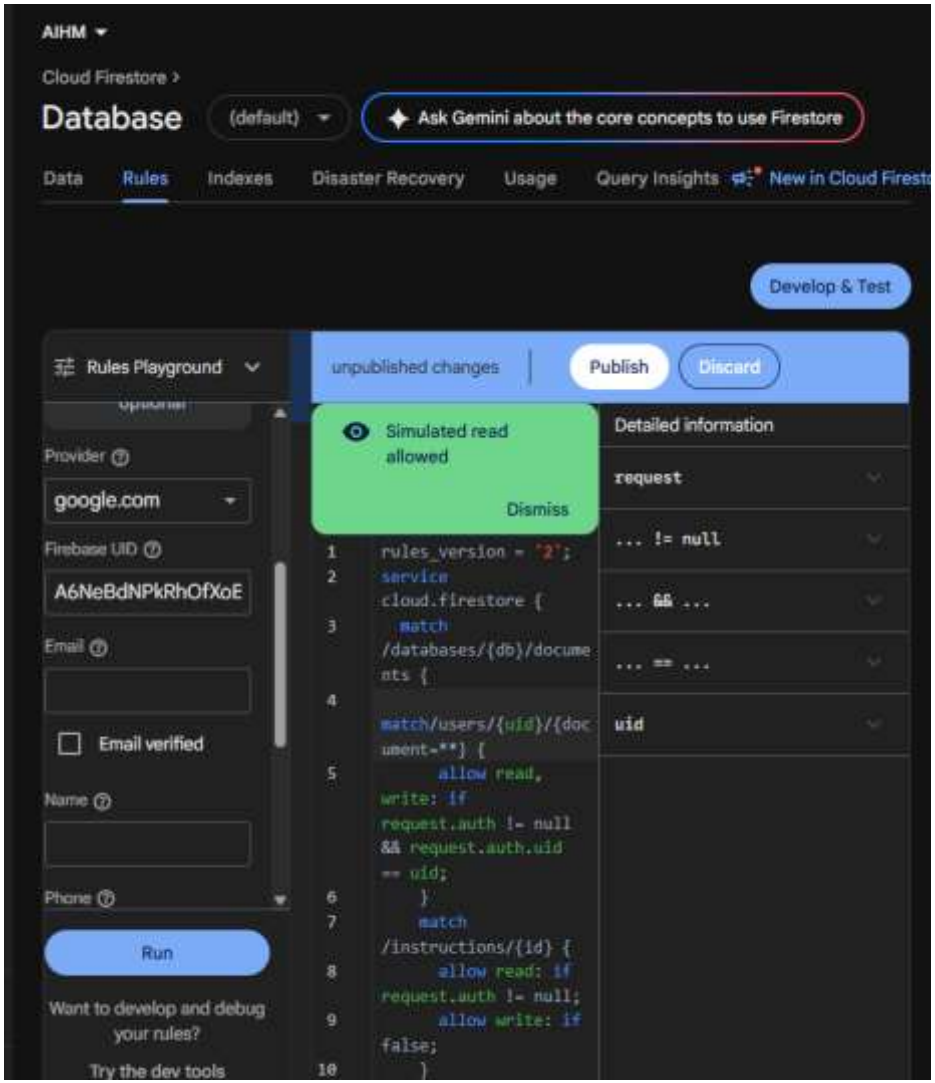
Figure 7: Notification permission request shown after the user gives consent.



We tested the access rules in the Firestore Rules Playground. A read request to a user's document from a different user was denied, while the same request from the owner was allowed.

Figure 8: Firestore Rules Playground showing access denied for a different user and allowed for the document owner.



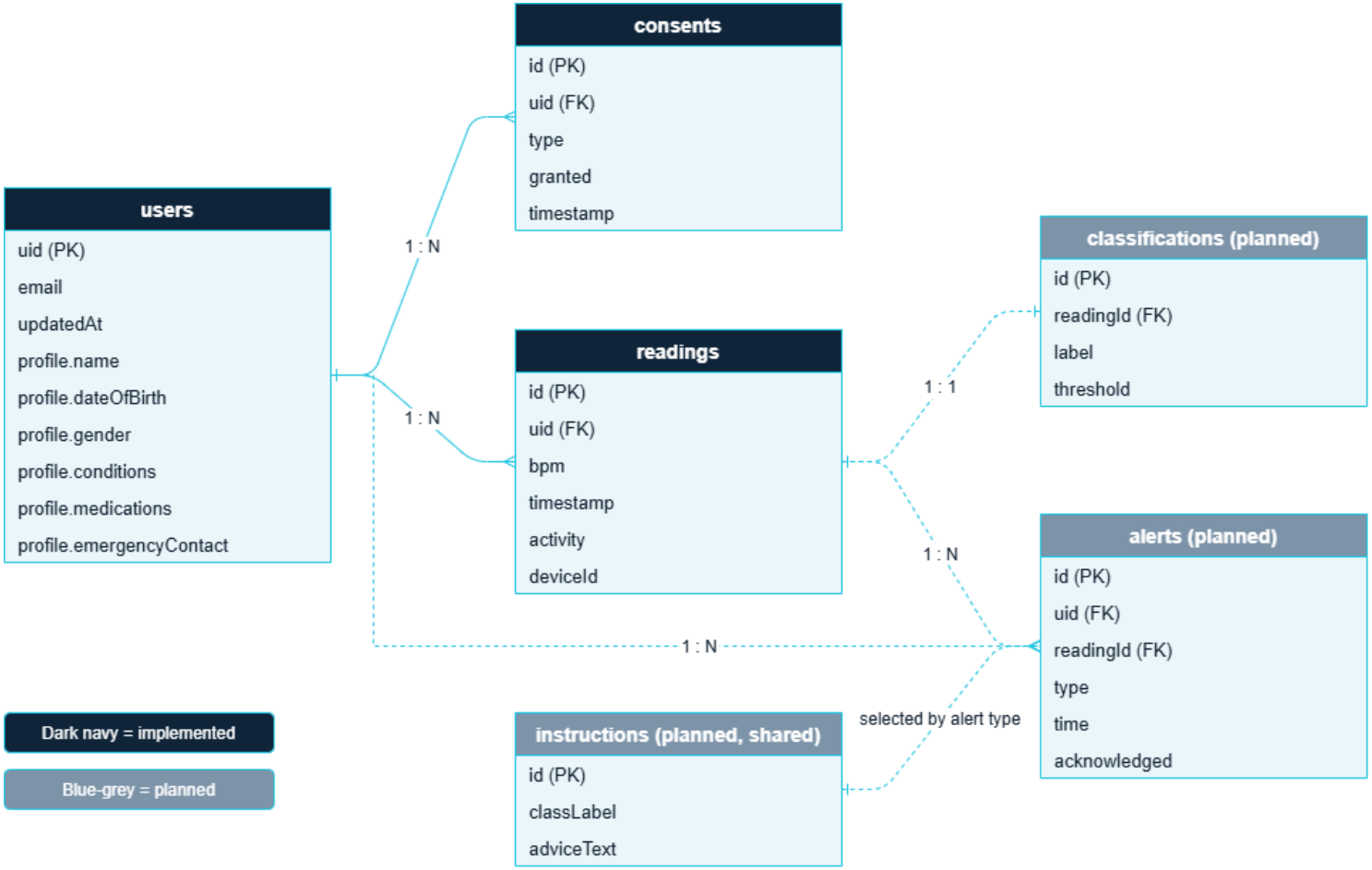


Restricted - مقيد

Task 5: Database Development

We used Cloud Firestore as the project database. Each user's data is stored under a document in the users collection, identified by the user's Firebase Authentication ID. The health profile is stored inside this document, and the consent records are stored in a consents sub-collection. Heart rate readings are stored in a readings sub-collection under the same user. Classification results and alerts will be stored in sub-collections under the user as well, and a shared instructions collection will hold the general health guidance.

Figure 9: Entity-relationship diagram of the AIHM database. Collections with a blue-grey header are planned and not yet implemented.



Restricted - مقيد

To control access, we wrote Firestore security rules. A signed-in user can read and write only the documents under their own ID, and no user can modify the instructions collection, which is read-only. All communication between the app and Firebase uses HTTPS. We tested the rules in the Firestore Rules Playground: a read request from a different user was denied, while the same request from the document owner was allowed ***(Figure 8).***

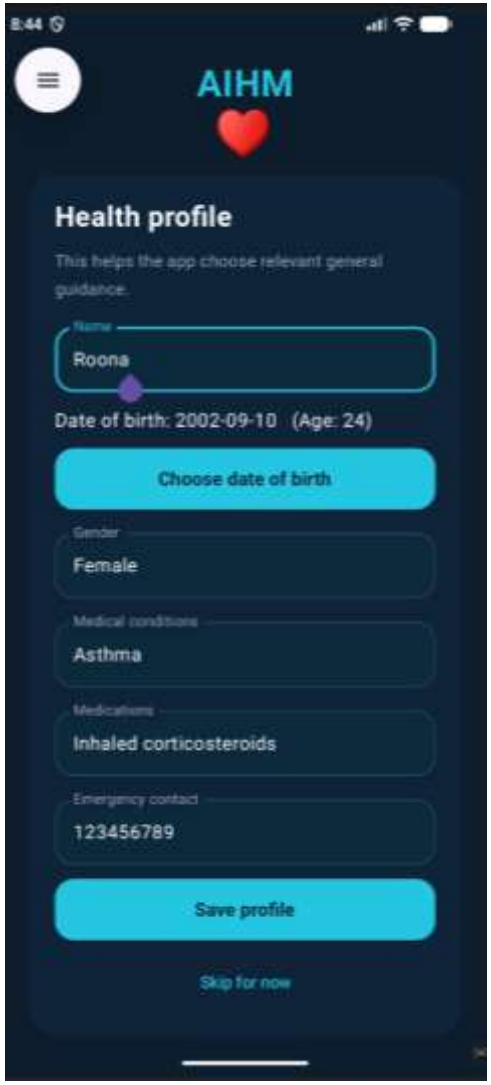
We also tested saving, retrieving, updating, and deleting data through the app.

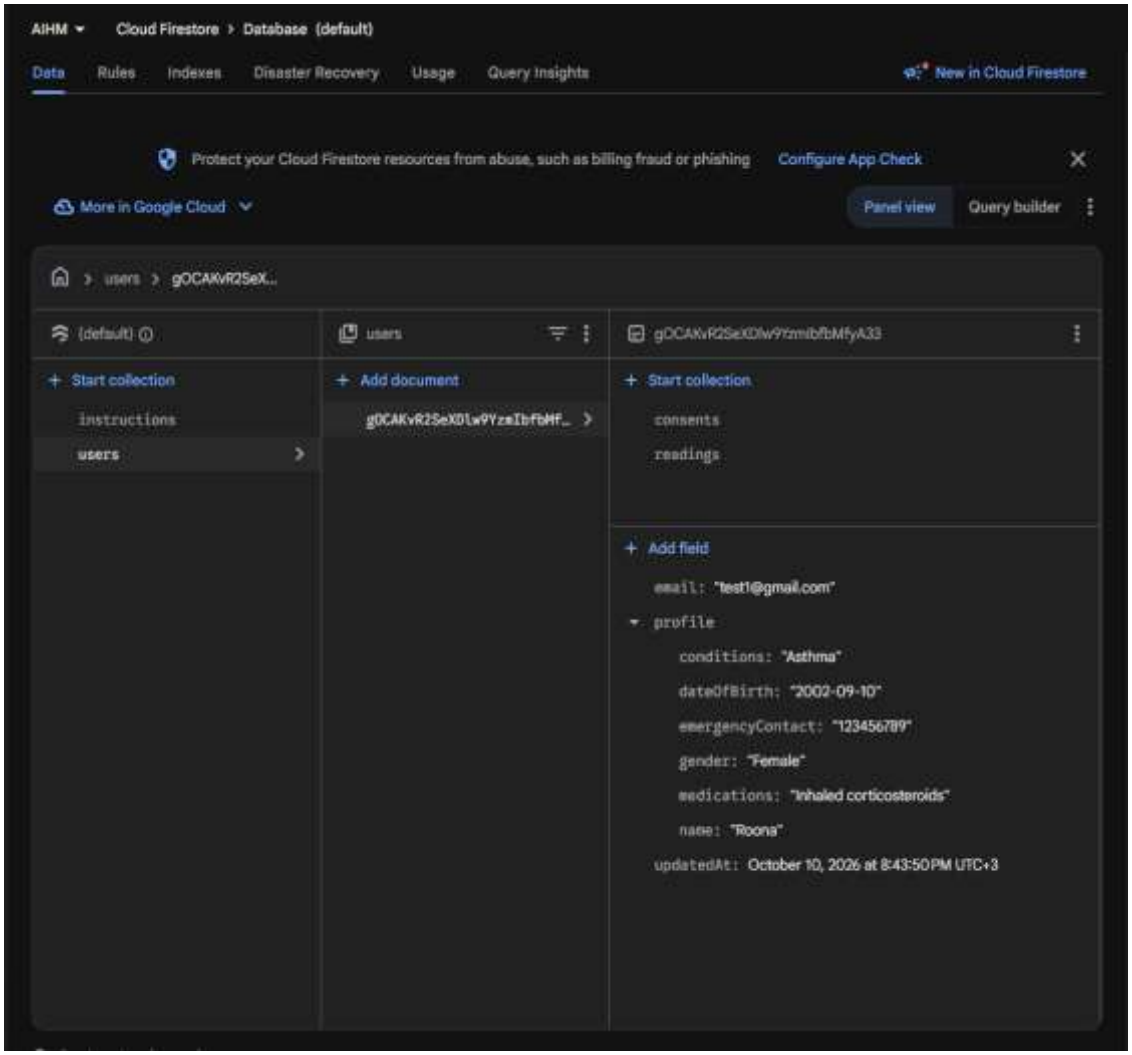
Table 1: Database test results

Test	Action	Expected result	Result
Save	Save a new health profile	Profile appears in Firestore under the user's ID	Passed
Save	Accept the consent statements	Two consent records with timestamps are created	Passed
Save	Receive a simulated reading from the watch	A reading document with bpm, timestamp, activity, and deviceId is created	Passed
Save	Save a profile with a date of birth	dateOfBirth appears in Firestore and the age is calculated from it	Passed
Retrieve	Open the Profile screen	Saved values load into the form	Passed
Update	Edit the profile and save again	Firestore shows the new values and a new updatedAt	Passed
Delete	Delete the account with the correct password	User, profile, and consents are removed	Passed
Delete	Delete with a wrong password	Request rejected, nothing deleted	Passed
Delete	Delete an account that has saved readings	Readings are removed along with the user, profile and consents	Passed
Access control	Read another user's document	Access denied	Passed
Access control	Read own document	Access allowed	Passed
Validation	Save with an under-18 date	Rejected with a message	Passed

Restricted - مقيد

Figure 10: Updated health profile in Cloud Firestore.

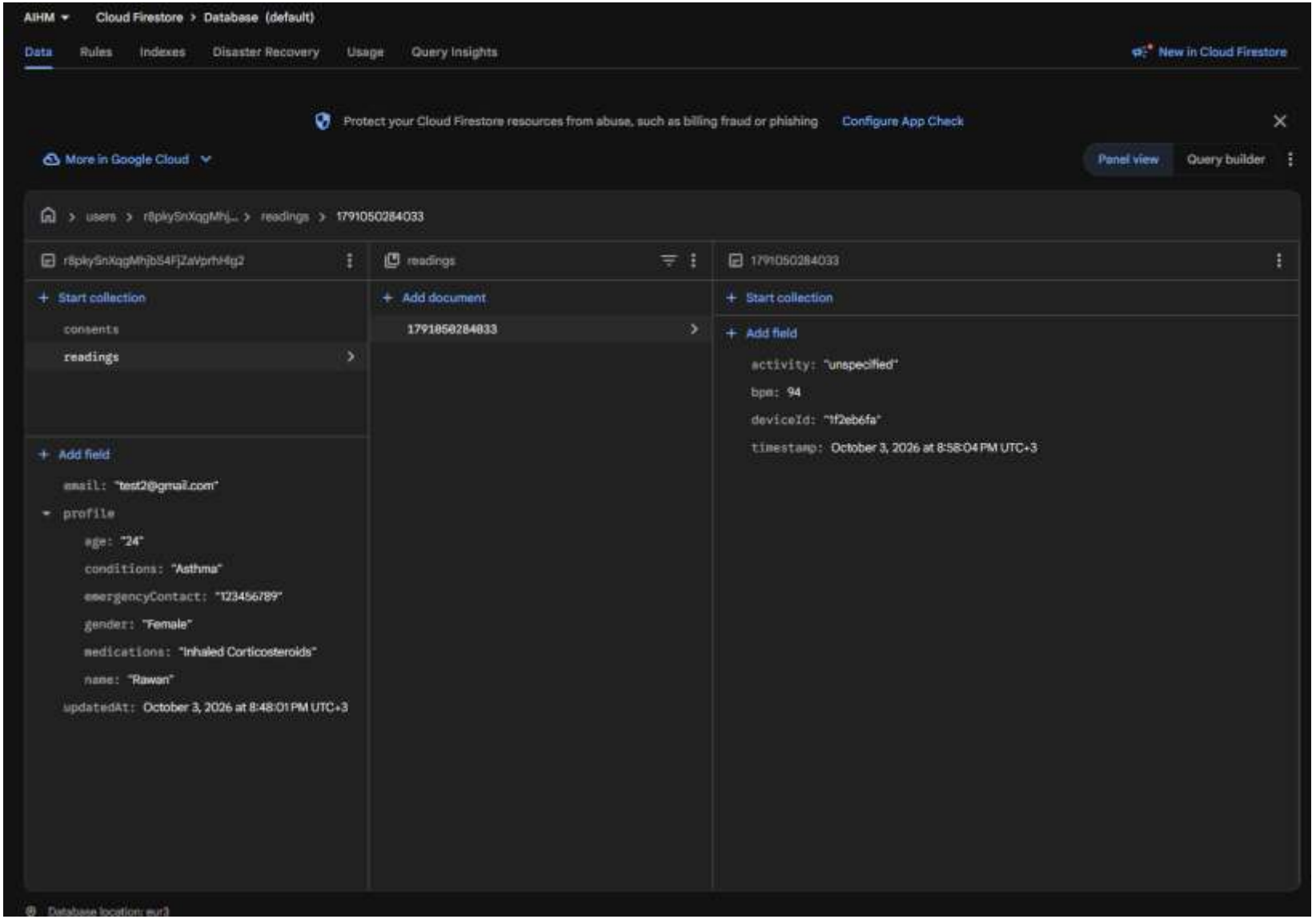




So far, the database stores profiles, consent records, and heart rate readings. The collections for classification results, alerts, and instructions are defined in the schema and covered by the same security rules, and they will be filled by the model (T13), the alert logic (T14), and the instruction library (T15) as those tasks are completed.

Restricted - مقيد

Figure 11: Heart rate reading saved in Cloud Firestore.



Heart rate readings received from the smartwatch are saved in a readings sub-collection under the user's account. Each reading stores the heart rate, the measurement time, the selected activity, and the ID of the source device. To stay within the free Firestore usage limits, the app saves at most one reading per minute. The values currently come from the watch app's test generator, so they are placeholders and not real heart rate measurements.

Restricted - مقيد

AIHM Database Schema Review and Suggested Improvements...

Name: *nofe Abdulrahman*

AIHM Database Schema Review

After reviewing Figure 9 and the Firestore screenshot in Figure 11, I noticed a few details that could make the design clearer. These are suggestions and points to check, not confirmed errors in the code.

1. Clarifying the Identifiers

My observation → The diagram shows uid and id as fields inside readings. In the screenshot, the user is identified through the document path, and the reading is identified by its document ID.

My suggestion → Explain this in the diagram so it matches how the data is stored. We do not need to repeat these identifiers inside the document just to match the drawing.

2. Saving Readings Before Classification

My observation → The diagram shows one classification for each reading. However, a reading may be saved before the model finishes classifying it.

My suggestion → If each reading is classified once, allow it to have zero or one classification. This means a reading can stay in the database while it waits for its result.

3. Separating New Alerts from Repeated Notifications

My observation → Sending another notification for the same alert does not always mean that a new event has happened.

My suggestion → Define when the app should create a new alert and when it should only repeat a notification. This would help prevent the same event from appearing several times in the alert history.

Restricted - مقيد

4. Keeping the Original Instruction Text

My observation → If an instruction is changed later, an old alert might show the updated text instead of what the user originally saw.

My suggestion → Save a copy of the displayed instruction in instructionTextSnapshot inside the alert. This would keep the original text available when the user opens an old alert.

5. Distinguishing No Answer from No Symptoms

My observation → A user who has not answered the symptom question is different from a user who has answered that they have no symptoms.

My suggestion → Make this difference clear in the saved data. A missing answer should not be treated as “no symptoms,” especially when the answer affects which instructions are shown.

6. Explaining the threshold Field

My observation → The classifications collection includes threshold, but its meaning is not clear in the diagram.

My suggestion → Explain whether it is a heart rate limit or a threshold for the model’s anomaly score. Its meaning should match the method used by the model.

7. Checking That Linked Records Belong to the Same User

My observation → An alert is linked to a user and a reading. We need to make sure these links are consistent.

My suggestion → Check that the reading belongs to the same user as the alert. This needs to be verified in the code because the diagram alone cannot confirm it.

Restricted - مقيد

The suggestions about classifications, alerts, and instructions are for features that are still planned. They do not mean that the completed work is incorrect. The next step is to review the relevant code and Firestore security rules to check how these relationships work in practice.

Name: *Abrar and Lyan*

1. Overall Evaluation

The schema of the AIHM database is a suitable basis for the development of a heart-rate monitoring app. This app makes use of Cloud Firestore as a place to store user profiles, consent records, and heart-rate readings. Moreover, the schema specifies the planned collection of classifications, alerts, and health instruction.
In general, the existing database schema is capable of covering the planned functionality of the app. Nevertheless, some features of the schema need to be explained in order to guarantee its capacity to support the mentioned above features.

2. Relationships and Data Organization

The presented schema shows the one-to-many relationships between users and readings, as well as between users and consent records. Thus, a user can have several heart-rate readings and distinct consent records in order to give permission to different actions.
Moreover, there are relationships between readings, classifications, and alerts. The mentioned relationships are needed for the fact that the application needs to link a particular classification or alert to the appropriate heart-rate reading.
Because Cloud Firestore is a NoSQL database, such relationships are represented using document paths and references and not relational foreign keys in a relational database. Consequently, it would be essential to ensure that the references are verified, and the records related to them belong to the same user.
One suggestion would be to clarify the relationship between readings and classifications. In the provided diagram, the relationship between the two is depicted as a one-to-one relationship. Such an approach is okay if every reading gets a single classification, but there must be provision for invalid readings.

3. Heart-Rate Readings and Data Reliability

The readings table includes columns like bpm, timestamp, activity, and deviceId. It can be used as a good starting point for recording heart rate measurements.
However, according to the project documentation, the readings at the moment represent simulated data generated by a test generator and not actual heart rate measurements.

Restricted - مقيد

One potential enhancement that could be applied here would be the possibility to distinguish between the simulated and real measurements if they are going to be stored in one database. Moreover, the application needs to validate mandatory fields, timestamps, and heart rate values prior to using them for classification and alerts.
These actions would increase the reliability of the data and would avoid treating the invalid test data as actual measurements.

4. Classification Results

The proposed classification table will contain fields such as readingId, label and threshold. This set of fields can be used as a basis for storing the classification information associated with a certain heart-rate reading.
It is important, however, to make the definition of threshold field clear. The project should describe the way classification borders are defined and the treatment of the invalid or unclassified readings by the classification algorithm.
The link between the readings and classifications should also be consistent with the actual implementation. If each reading is classified separately, then there is no need to change the schema – one-to-one relationship will suffice. In case when the model classifies groups of readings in a certain period of time, this period should be represented in the schema.
In addition, storing the timestamp of the classification and the model version used could become useful for debugging purposes. These features are optional extensions rather than confirmed requirements of the current implementation.

5. Alert Generation and Management

The proposed alerts will consist of attributes like readingId, type, time, and acknowledged. These attributes give a good outline on how alerts and acknowledgement of alerts are going to be recorded.
However, the project requirements say that the alert logic will depend on the persistence of abnormal readings and contextual data. It means that there must be a way of showing how the alert and the reading or the event leading to the alert are related.
If the logic of alert depends on a number of readings within a particular time frame, the schema will have to represent the time window or triggering event. In case of individual readings triggering the alert logic, then the existing reference will suffice.
There should also be logic on how multiple abnormal readings are treated. This will help avoid unnecessary duplication of alerts. The introduction of the start and end time fields for the alerts should only be done if necessary by the logic of alert generation.

6. Instructions and Personalized Guidance

The proposed shared instructions collection will hold classLabel and adviceText. Maintaining general instructions in the shared instructions collection can reduce duplication and simplify maintenance of the instructions.
But the project guidelines state that the guidance needs to be chosen based on the type of alert, the reported symptoms, and the profile data. Hence choosing the instructions just based on the classification may not be enough.

Restricted - مقيد

The application needs to describe the selection criteria for selecting the instructions and how these selected instructions are linked to the alerts.
If the application needs to store the exact instruction text seen by the user during alert generation, then the text can be stored as part of a snapshot of the instruction. This is not an enhancement but an optional enhancement.

7. Consent, Security, and Account Deletion

Consent is stored in a separate consents subcollection for each user. It allows us to meet the condition that requires separate consent for storing health data and heart rate monitoring.
Storage of consent type and timestamp helps us understand what kind of consent was received and when. The consent version can also come in handy in case the consent language changes at some point in the future.
The access control mechanisms have already been tested in our project. They prevent one user from accessing the document of another user. It is an essential security measure since the app works with private and health information.
The testing of account deletion proves that the currently stored user data, consents, and readings can be deleted. Once we implement the collections of classification and alerts, we need to test it once again.

8. Consistency Between the Diagram and Implementation

One of the advantages of the project documentation lies in the fact that it divides collections into implemented collections and those that are supposed to be implemented. The Profile, Consent, and Reading collections have already been implemented, whereas Classification, Alerts, and Instructions collections are still under planning.
As a result, such collections cannot be regarded as failed implementations. It is necessary to evaluate whether their proposed structure is suitable to meet the requirements.
It is also necessary for the team to monitor the consistency of the database schema, Firestore security rules, application logic, and test results as more features are added.

9. Recommended Improvements

The following improvements could strengthen the database design:
1. Clearly define the relationship between readings and classification outcomes, including treatment of unreadable or invalid readings.
2. Make sure heart rate, timestamp, and source validation is done before using readings for further processing.
3. Define classification thresholds and their application.
4. Make sure alerts can be linked to specific readings and events in cases where abnormal conditions are ongoing.
5. Define alerting conditions and notification reoccurrence.

Restricted - مقيد

6. Make sure instruction selection takes into consideration alert type, symptoms and profile data.
7. Ensure account deletion deletes all user-related data in case of planned collection implementation.
8. Make sure that the database scheme reflects the implementation and differentiate between currently implemented features and planned features.
Optional fields such as isSimulated, modelVersion, and consentVersion should be introduced only when they serve a clear purpose in the application.

10. Conclusion

On balance, the AIHM database schema constitutes a sufficient basis for the current implementation of the project and its future functionalities. The way the database schema is designed to store users' profiles, their consents and heart rate measurements covers the minimum required functionality.
The possible areas for improving the database are those related to tracking the classifications' outcomes, handling the alerts, choosing personalized health recommendations and protecting the user data during system evolution.
It is not required to design a completely new database schema as per the evidence available at hand. It is more appropriate to specify the relationships needed to implement new functionalities, to verify input data, to document the process of classification and alert creation and to test security and deletion procedures as more collections are added

Restricted - مقيد